



SUPPLY-CHAIN SECURITY

Keyless Image Signing & Attestation

Every image is signed and attested in CI with **no stored key** — trust is anchored to *who* built it — and re-verified at admission before it can run.

GitHub OIDC

Fulcio

Rekor

cosign

Ratify

Gatekeeper

Trivy

1 Why keyless — trust “who built it”, not a stored key

no durable secret to leak

 **Key-based signing**

A **long-lived private key** you must store, rotate and protect. Leak it and anyone can forge signatures — the classic `invalid pem block` class of pain.



 **Keyless signing**

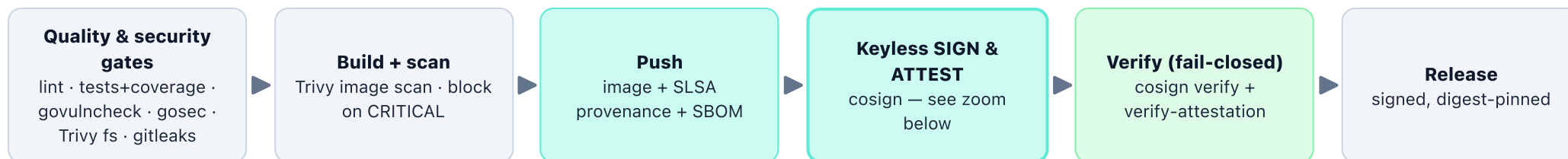
No key at all. A CA issues a **~10-minute certificate** bound to the builder's **identity** (the GitHub workflow), the image is signed, and a public ledger records it forever.

THE PIECES THAT MAKE IT WORK

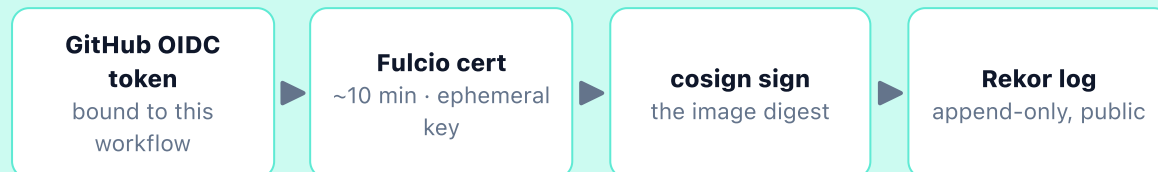
Component	Role
GitHub OIDC	Vouches for <i>who</i> is running the job — a signed identity token bound to the exact workflow (<code>token.actions.githubusercontent.com</code>).
Fulcio	Sigstore CA — exchanges a valid OIDC token for a short-lived X.509 certificate binding an ephemeral key to that identity.
Rekor	Public, append-only transparency log of every signature & attestation — tamper-evident, publicly auditable.
cosign	The client that orchestrates <code>OIDC → cert → sign → log</code> and performs verification against an identity.
Ratify + Gatekeeper	In-cluster verifier — checks the signature & attestations at admission and denies anything untrusted.

2 Signing & attestation — the CI pipeline

build → sign → attest → verify (fail-closed)



Zoom: the keyless sign (no `COSIGN_KEY` anywhere)



The ephemeral key is discarded immediately — nothing durable is stored. Signer identity = `github.com/<repo>/github/workflows/release.yml`, issuer = `token.actions.githubusercontent.com`.

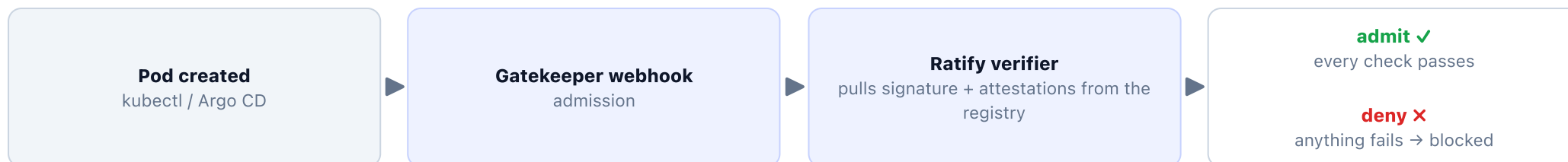
What gets attested

`cyclonedx · SBOM` `vuln · Trivy report` `quality · coverage`
`SLSA provenance`

Each is a **signed attestation** on the same digest — machine-checkable evidence that travels *with* the image, not a wiki page.

3 Verification — nothing untrusted is admitted

re-verified at the cluster door



✓ What Ratify checks

- Signature is valid & recorded in **Rekor**.
- Certificate **identity** == the trusted GitHub workflow.
- OIDC **issuer** == `token.actions.githubusercontent.com`.
- Required **attestations** (SBOM · vuln · quality) are present.

Roots for Fulcio/Rekor come from the Sigstore **TUF** trust root — verification needs no stored public key, just the pinned identity.

🧩 Complementary admission gates

`allowed-repos` `supply-chain-evidence` `image-quality`
`disallow-latest / require-digest` `rollout-only` `trivy-vulnerabilities`

Gatekeeper (image-policy) and Trivy add checks alongside Ratify — provenance/evidence present, pinned digests, CVE thresholds — so signature *and* content are gated.

Same identity asserted at sign time is required at verify time — **no shared key**, on either side. (Signature enforcement is scoped to the `ratify-demo` namespace today; the model generalises to all tenant namespaces.)

🔑 At sign time (CI asserts)

```
--certificate-identity
https://github.com/.../release.yml
--certificate-oidc-issuer
https://token.actions.githubusercontent.com
```



🔒 At verify time (cluster requires)

Ratify's trust policy pins the **same** identity + issuer. A signature from any other workflow, repo or issuer — however valid — is rejected. The match *is* the trust.

Rotate nothing, store nothing: change the trusted identity and the anchor moves with it.

FROM ATTESTATION TO A BADGE THE DEVELOPER SEES

Signed attestations

SBOM · vuln · quality · provenance



teams-api

reads the evidence (app-compliance)



portal badge

per-app supply-chain compliance



Fail-closed twice — the pipeline re-verifies before release, and the cluster re-verifies before run. A broken signature stops the build *and* can never be admitted.



Publicly auditable — every signature/attestation is in Rekor; anyone can independently verify an image with one `cosign verify`, no platform access needed.



Nothing to steal — no signing key exists at rest; compromise of CI secrets can't forge a past or future signature.